Git Command Line Interface

Git is the most popular version control software in the software industry. For an individual, it carefully records each change you commit to files in a repository, backs up your files for you, enables you to manage a single project from many machines, and can be used to automatically run a routine each time the remote repository is updated. For a group of collaborators, it also allows you to work together by keeping a **main** version of a project updated with changes implemented by different users on different branches of the repository by merging commits made asynchronously on different machines. These are only some of the things git can do, making it an essential tool for anybody interested in software development or group projects in general.

There are various graphical user interface (GUI)s which handle git, including packages provided by github and/or built into integrated development environments (IDEs; E.g. Visual Studo Code) which provide interfaces for git apart from the terminal. In this module, we will not be using those tools, and will learn how to manage repositories using the command line interface (CLI). Furthermore, it is reccomended that you do not use a terminal built-in to an IDE, and instead use the terminal built into your operating system. This guide will be most useful for UNIX based operating systems (E.g. MacOS, linux).

Setup

Most UNIX based operating systems (E.g. MacOS, linux) have the git package installed by default. You can test this with the command:

```
git --version
```

Beyond having git installed on your computer, you also need to have an account at a hosting website such as github or gitlab In this guide, we will be using gitlab, so be sure to make an account.

You will also need a web browser and text editor. You may use an IDE to edit files, but when I ask you to close a file, I need you to completely close out of your IDE or text editor. It is very important that we save and close files when the instructions ask you to do so. Leaving them open in an IDE or text editor may cause issues.

Optional -- Secure authentication

The easiest way to authenticate with git is a username and password. However, this is not the most secure. A better way is to use two-factor authentication and an SSH key.

Two factor authentication
Gitlab's instructions for setting up 2FA
Abridged:

1. Download an authenticator app on your smartphone (E.g. google authenticator).
2. Navigate to The appropriate settings page
3. Generate a code or barcode, and read that code into your smartphone authentication app.

SSH Keys
gitlab's instructions for using an SSH key
User settings / SSH keys

SSH is a secure protocol which generates a pair of public and private keys. The public key is like a math problem, and contains no sensitive information. It can be shared or freely passed around on the internet with little to no risk. The private key is stored on your device, and represents the solution to that math problem. SSH allows you to connect to a device on the internet which has a copy of your public key by solving the math problem by knowing the solution (the private key). Brute-forcing the math problem is effectively impossible with modern computer hardware.

Using an SSH key to authenticate when connecting to gitlab through the command line is secure, and does not require 2FA (however, you can't use an SSH to connect to gitlab through a web browser; only through the command line).

SSH should be installed on your machine already. Very few operating systems come without some kind of SSH installation. You can verify this with the command:

```
which ssh
```

Setting up gitlab with an SSH key:

1. On a new terminal, navigate to your SSH folder:
   `cd ~/.ssh`
2. Generate an SSH key pair:
   `ssh-keygen -t rsa -b 2048 -C "This is my gitlab key"`
   When prompted, save the key in:
   `~/home/user/.ssh/gitlab_rsa`
   Replace user with your username. If you are uncertain of your username, you can find it by entering:
   `echo ${HOME}`
   You may wish to specify a password for your ssh key. This is reccomended if you are using the key on a computer which other people use.
3. Optionally, open your `~/.ssh/config` file in your favorite text editor

   Add this entry:

   ```
   HOST gitlab
       HOSTNAME gitlab.com
       User git
       IdentityFile ${HOME}/.ssh/gitlab_rsa
       IdentitiesOnly yes
   ```

   where `${HOME}` is replaced by your home directory (for me that's `/home/xevra` on my linux machine and `/Users/vdelfave` on my MacOS machine).
4. Navigate to User settings / SSH keys
5. Select "Add new key"
6. Copy the contents of your public key to you clipboard. One way of doing this is by executing the command:
   `cat ~/.ssh/gitlab_rsa.pub`
   and then selecting the output in your terminal and copying it to your clipboard with the mouse (by right clicking).
7. Paste the contents of your public key into the "Key" box
8. Enter a title (You can use your computer's hostname to keep track of keys; E.g. My tower is called Temerity, and my macbook is called Maruna, and I have different SSH keys for each device).
9. Save the key

Definitions

General

Repository:
The place where your code is stored. A directory (folder) with your project files in it, which has been initialized through git.

Local:
On your machine. The your local repository is the folder on your computer where your git project exists.

Branch:
An offshoot of the repository. A newly created repository will have a single branch (usually `main` or `master`).

Tag:
A label associated with a particular branch.

Remote:
The place where a repository exists on the internet. This would usually be referenced by a URL associated with the website where your repository is managed (E.g. github or gitlab).

Fork:
A remote which tracks another remote. For example, if I use the web interface to create a copy of a repository managed by a group, my copy is a fork of the remote managed by that group.

Upstream:
When we discuss an "upstream" branch, we are usually talking about the online branch which a local branch tracks. Generally, the upstream branch is the more trusted branch. For example, a branch with the tag "main" would usually be upstream of a branch with the name "development".

Origin:
An online repository which is tracked by a local repository. Git allows you to track branches from several remotes in a local repository, but only one of these remotes is considered the origin (usually the remote which you initially cloned).

URL (Uniform Resource Locator):
a web address that provides a unique, specific location for a particular resource on the internet. A URL can have several parts, including (but not limited to): i) the protocol (http, https, ssh, etc...) used to access a resource; ii) the hostname (or domain name); iii) the path of a resource at a given domain.
See Github's documentation on URLs of remote repositories
For our purposes, the path remains the same whether we access a remote repository through the https or ssh protocol. A github or gitlab url using the https protocol will look like:
`https://gitlab.com/user/repo.git`
An SSH url may look like:
`git@gitlab.com:user/remote.git`
or (if you edited your ssh config file to associate the USER "git" and HOSTNAME "gitlab.com" with the HOST "gitlab"), it might look like this:
`gitlab:user/remote.git`

Commands

This is not all of the git commands; only those we will use in this lesson's activities. Note: to read the manual pages for any of these commands, you can enter `man git-[command]` into the terminal.

git clone [url]:
Download the repository from the url and create a local repository, with the online repository as its origin. The url can use https or ssh protocol (See url).

git config [options]:
Can be used to configure several options within git. For example, you can provide a name and email to sign your commits with using `git config --global user.name FIRST_NAME LAST_NAME` and `git config --global user.email FIRST_NAME LAST_NAME`. The `--global` option sets the configuration for all of your repositories, instead of just one.

git status:
Check the status of a repository. This will list files in your local repository which have uncommitted changes.

git add [targets]:
Update the index of your current branch in a local repository with changes to selected files. To add all modified files at once, run `git add --all`. This option also registers the deletion of files which were deleted without a `git rm` command.

git commit:
Create a new commit to the current branch in your local repository. Commits require a commit message, and this can be done in the command line with `git commit -m "message"`

git rm [targets]:
Remove a file or set of files from the hard disk, and simultaneously update the index for the local branch. Without doing this, using a `rm` command will not update the local branch index.

git push:
Merge changes from your current local branch upstream (to the remote repository which the local branch tracks). This will backup changes to any files which have been committed on the local branch.

git pull:
Merge changes from the upstream branch of a remote repository into the local branch. This will update the files on your

machine with changes registered online.

git reset --hard [commit]
Change the current local branch to track changes from a particular commit. This will delete any changes you have made on your machine which are uncommitted. If you mess up your local branch, I mean, really mess it up, you can use this command to start over from any commit.

git branch:
Check which branch you are currently on, and also list branches tracked in the local repository. Use `git branch -vv` to also list the upstream branch tracked by various local branches and their associated remote repository.

git checkout [tag]
Switch the currently active branch to the branch labeled [tag]. If you checkout a branch which has not been used before in the local repository, git will attempt to load the branch from the origin repository. If you want to create a new branch labeled [tag], use `git checkout -b [tag]` . When you push a new branch which has not been registered in the remote repository, you will need to use `git push --set-upstream [remote] [tag]`, where `[remote]` is usually "origin"

git remote [options]:
You can use `git remote add [remote] [url]` to track an additional remote for the repository. You would want to do this, for example, if you want to include branches from multiple forks of the same repository in your local repository. The identifier [remote] can be anything, so long as you remember it. It plays the same role as the keyword "origin" in many git commands.

You can also use the command `git remote set-url [remote] [url]` to update the upstream url which a local branch tracks. You would want to do this if you want to switch from using the `https` protocol to `ssh` protocol, or if you copied an entire repository to a new machine using a flash drive. `set-url`

git fetch [options]:
You can use this command to update the information git has available in your local repository with information from one or more remote repositories. Any time you pull from a remote repository, git will run this command first. You can also use this command to add branches to the local repository which occur in a remote repository. To track a single branch from a remote repository, you can use `git fetch [remote] [tag]` .

git merge [tag]:
Merge changes from a local branch, [tag], into the active working branch. I generally do this through gitlab's web interface, but whenever there are complicated merge conflicts, you need to do this locally.

Lesson

Activity 1:

In this exercise, we will create a new repository through gitlab's web interface, and use the command line to download and change files in that repository.

Part 1:
Navigate to gitlab.com, login, and click "New project" in the upper right-hand corner of the page. Select "Create a blank project." Name your project. We'll be using this repository for the lessons on other command line tools as well. I would reccomend "example", to indicate that this is an example of a repository, or "apps", since we will be working toward an automated pipeline for putting together job/grant applications. Select Private, and initialize repository with a README. Create project.

Part 2:
Add your instructor to the project by navigating to `https://gitlab.com/[username]/[repository]` in your browser, and using the toolbar to select "Manage" and then "Members". Select the "Invite members" button, and invite your instructor by gitlab username.

Part 3:
Open a terminal, and navigate to a folder where you would like to put files associated with these lessons. Reminder: I am asking you to use a terminal window which is not part of an IDE (like VScode). You may use an IDE to edit files, but when I ask you to close a file, I need you to completely close out of your IDE or text editor. It is very important that we save and close files when the instructions ask you to do so. Leaving them open in an IDE or text editor may cause issues.

In the terminal, clone your repository using the correct URL. If you have skipped setting up an SSH key, this will look something like this:

```
git clone https://gitlab.com/[username]/[repository]
```

where [username] is your gitlab.com username and [repository] is the name you just gave to your new repository.
If you set up an SSH key, and did not modify your `~/.ssh/config` file, your command may look like this:

```
git clone git@gitlab.com:[username]/[repository]
```

If you followed my intructions and updated your config file, this will be:

```
 git clone gitlab:[username]/[repository]
```

Part 4:
Navigate to your freshly cloned local repository, and list the files in that directory.

```
cd [repository]; ls
```

You should see a `README.md` file. Open it in a text editor of your choice, add some text (including your name), save the file, and close it.
Check the status of the repository using

```
git status
```

You should see (in red): `modified: README.md`.
Register the changes to `README.md` in the local index using:

```
git add README.md
```

Commit the changes to `README.md` in the current active branch using:

```
git commit -m "Added name to README.md"
```

Push the changes to the remote:

```
git push
```

This concludes the first activity.

Activity 2:

In this exercise, we will create a new branch in our local repository, and we will merge several commits into a main branch using the command line. We will walk through how to handle merge conflicts.

Part 1:
Open a terminal window, and navigate to the local repository we created in Activity 1. In order to check which branch we currently have active, execute:

```
git branch -vv
```

You should see that we are on the main branch, tracking origin. You should also see the most recent commit to the branch.
Create a new branch called "development" by executing the command:

```
git checkout -b development
```

Check again that we have switched to the development branch using:

```
git branch -vv
```

You will notice right away that we are not tracking any remote repository on our local development branch. We can push this branch to gitlab.com by using the command `git push origin development`. However, don't do this. Instead, we will make changes to the branch locally and merge it manually. Create a new file called "Makefile" by executing the command:

```
touch Makefile
```

Commit this file to the development branch:

```
git add Makefile
git commit -m "Created an empty Makefile"
```

Part 2:
Now, checkout the main branch and list the files in your current working directory:

```
git checkout main
git branch -vv
ls
```

You should not see the Makefile in your current working directory.
Now, we are going to merge changes from the development branch into the main branch:

```
git merge development
git branch -vv
ls
```

You should see that the main branch is now tracking the commit which we added to the development branch, and the Makefile is present in the current working directory.

Part 3: Creating a merge conflict
Checkout the develoment branch again, using:

```
git checkout development
```

Create a new file called `myscript.sh`, and open it in your text editor. Copy and paste the lines:

```
#!/bin/bash
echo cats
```

into the file, save, and close it. Test that you can run the script by executing:

```
bash myscript.sh
```

Commit the changes to the development branch, checkout `main`, and merge the changes into main.
Still on the main branch, open the file in your text editor, and modify the second line to be: `echo I like cats` Save, and close your text editor. Add and commit the changes directly to your main branch.
Checkout the development branch.
Open the text file again, and you should see it does not register the changes which were committed on the main branch. It should still read `echo cats`.
Edit this line to read:

```
echo I love cats
```

Save and close the file. Add and commit the changes to the development branch.

Part 4: Resolving a Merge Conflict
Checkout the main branch. Execute:

```
git merge development
```

This should fail. You should see:

```
CONFLICT (add/add): Merge conflict in myscript.sh
Automatic merge failed; fix conflicts and then commit the result.
```

Run:

```
git status
```

This should explain that we have unmerged paths.
We have created a merge conflict.
Now, open the file in your text editor. It should look something like this:

```
#!/bin/bash
<<<<<<< HEAD
echo I like cats
=======
echo I love cats
```

```
>>>>>>> development
```

We need to resolve this merge conflict by choosing one or the other. We need to delete four lines from this file, until the file reads:

```
#!/bin/bash
echo I like cats
```

or

```
#!/bin/bash
echo I love cats
```

If we choose the former, we are resolving the conflict by declaring that the main branch is right and the development branch is wrong. If we choose the latter, we are resolving the conflict by declaring that the development branch is right and the main branch is wrong. Choose, and let your heart decide. Once you are done, save the file and close your editor. Add and commit the changes. Push them.

```
git push
```

Activity 3:

In this exercise, we will create a fork of a repository through gitlab's web interface, make changes to it locally using the command line, and handle a merge on our local machine. We will then issue a pull request to the upstream repository of our fork using gitlab's web interface.

Part 1: Creating the fork
Navigate to cli-lessons in your browser. Select the "Fork" button. Keep all of the default settings.
Navigate to `https://gitlab.com/[username]/cli-lessons`, and verify that you have created the `cli-lessons` repository under your username. Clone from **your fork** of cli-lessons:

```
git clone gitlab:[username]/cli-lessons
```

Part 2: Making changes and causing problems
Navigate to the local repository in your terminal. Make sure you are on the main branch of your fork. Add your name to the readme, save and close your editor, add and commit that change. Push to your fork of `cli-lessons`
You will not be able to push anything directly to my fork of `cli-lessons`. You will have to create a pull request through gitlab's web interface. However, I have rules for my `cli-lessons` repository. I only accept pull requests to the **development branch** of `cli-lessons`. In addition, The `README.md` file on my development branch looks completely different than it does on main. It will not merge automatically.

Part 3: Cleaning up this mess
Your local repository is currently tracking only your fork of the repository, which is labeled `origin`. In order to merge with my development branch, you will need to merge changes from the development branch into your local main branch first. In order to accomplish that, you will need to add my remote:

```
git remote add xevra gitlab:xevra/cli-lessons
```

Now you need to fetch the development branch from that remote:

```
git fetch xevra/development
```

Now, look at the different branches in your local repository using

```
git branch -vv
```

and make sure that you are on the main branch, tracking your origin, and not on the development branch. Also make sure the development branch is tracking the development branch on my remote (xevra).
With that done, now merge the changes into main on your local branch:

```
git merge xevra/development
```

This will create a conflict. Resolve it in your text editor, and add your name to the list of authors. Push to your remote.

Part 4: Creating a merge request
Navigate to `https://gitlab.com/[username]/cli-lessons/-/branches` in your browser. Create a pull request from your main branch into my development branch. Tell me when you are done, and I'll review the pull request.